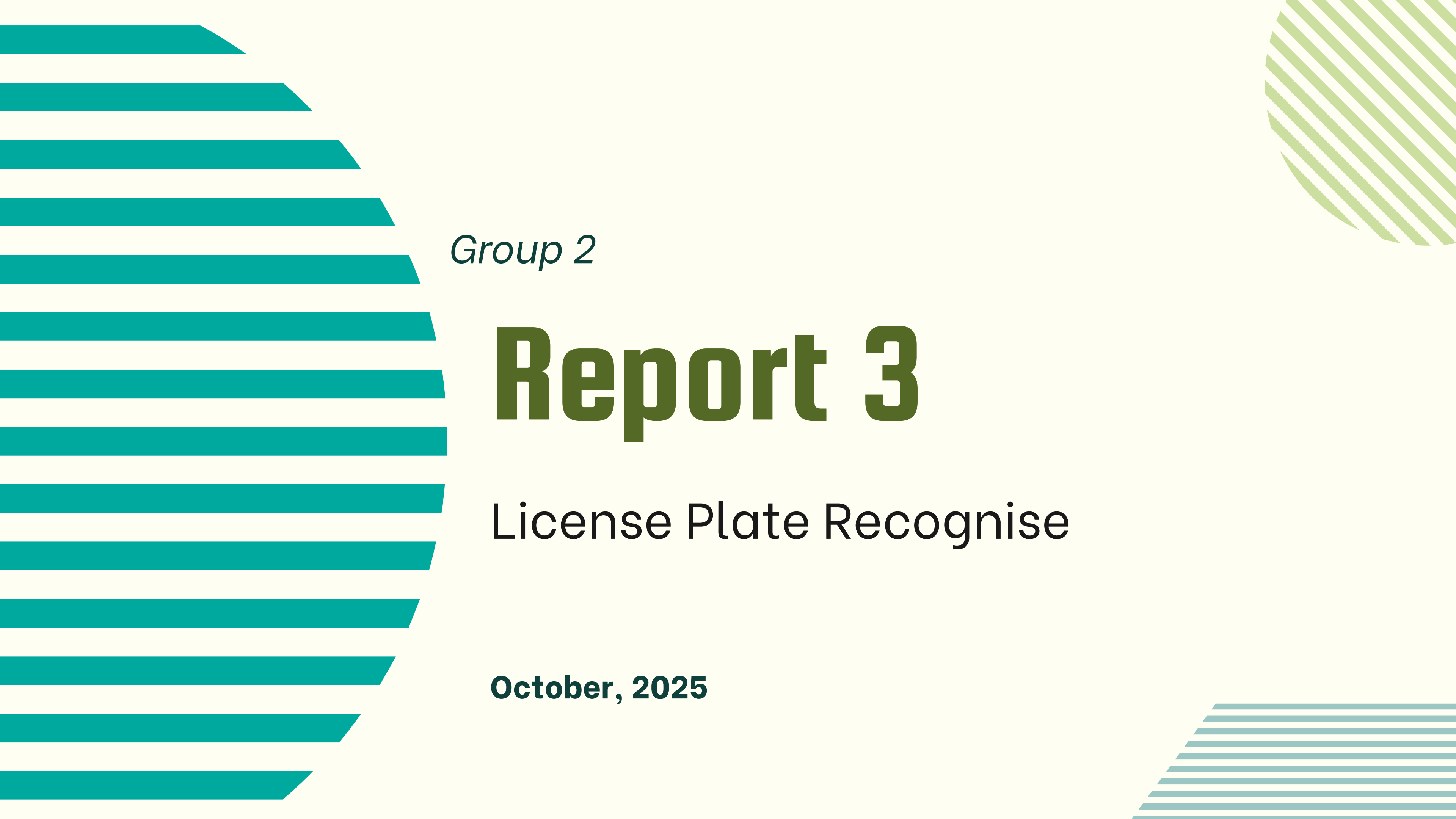


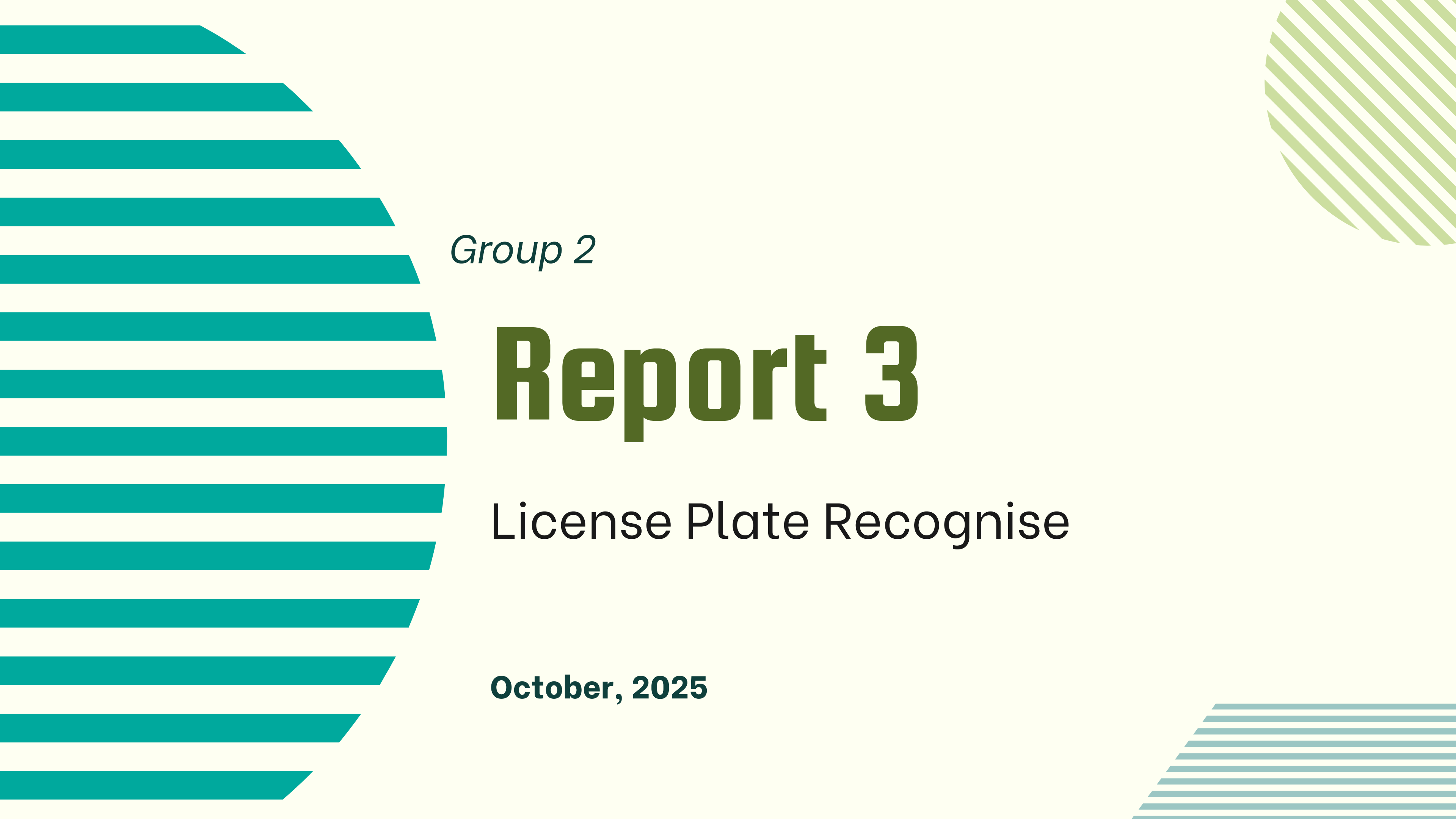


Group 2

Report 3

License Plate Recognise

October, 2025



Report

Highlights

01

Data Preprocessing

02

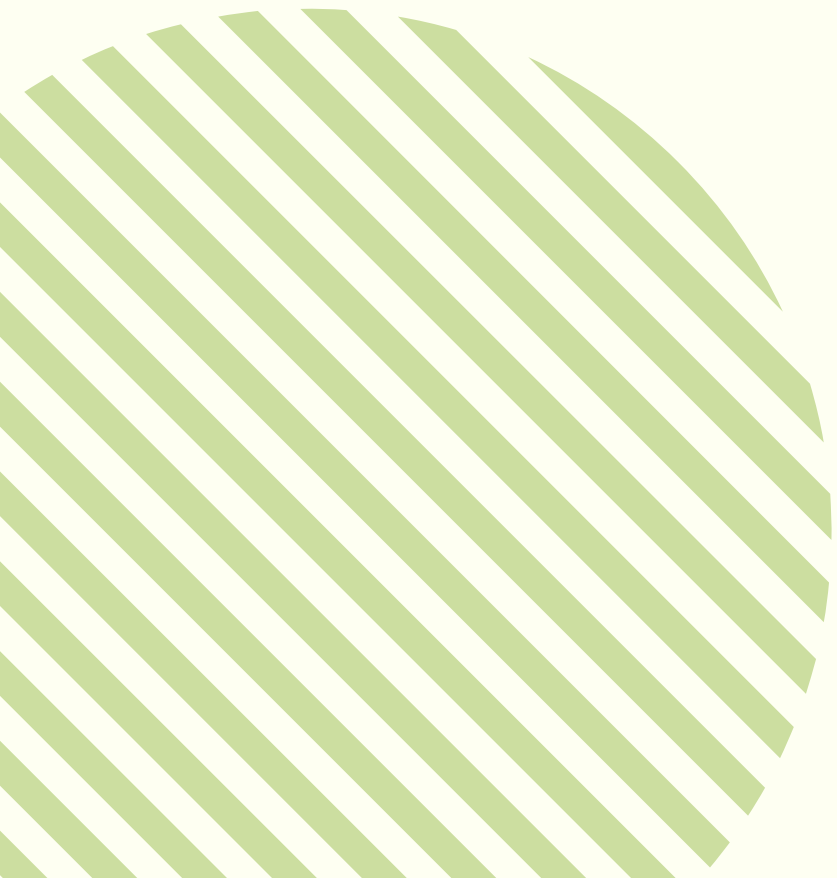
Metrics for evaluation

03

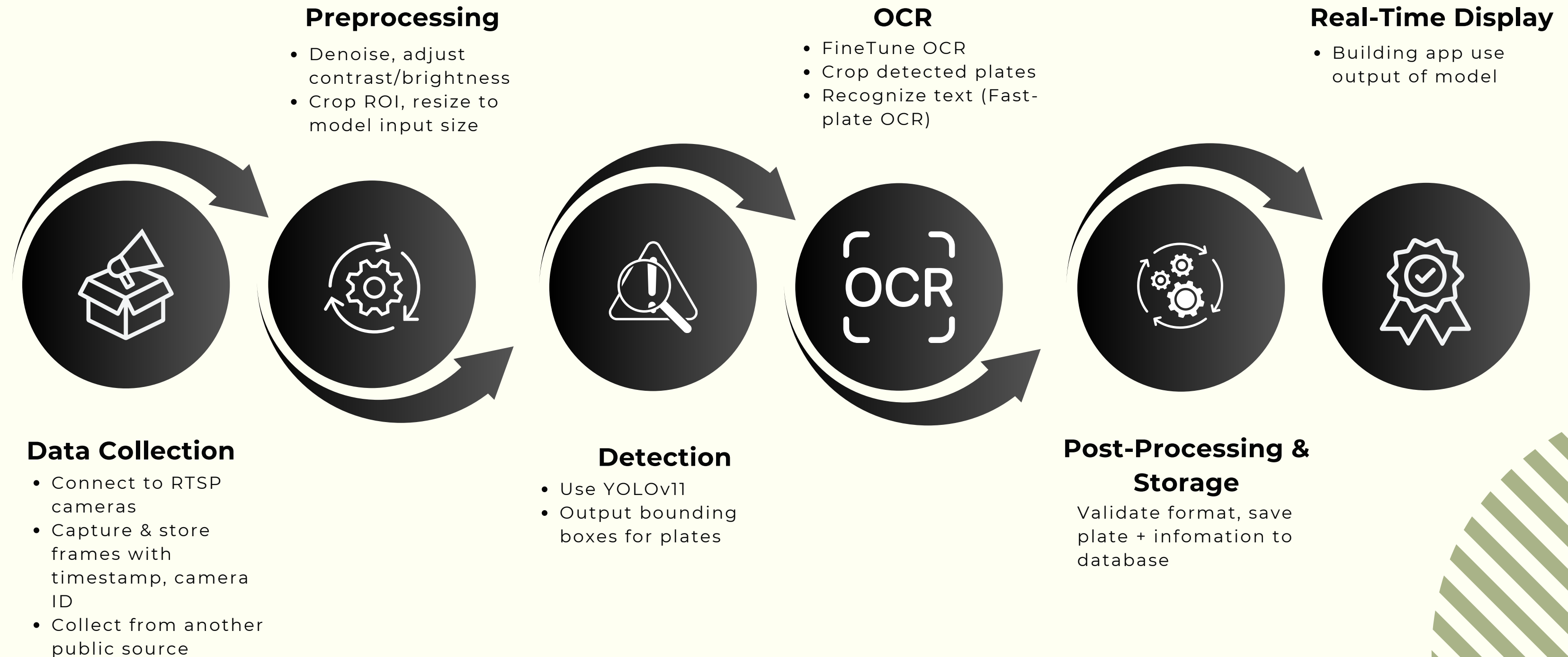
Summary models

04

Explain models



Overview Project



Data Preprocessing

How to crawling data

```
C:\Users\Lenovo>ffmpeg -rtsp_transport tcp -rtsp_flags prefer_tcp -i "rtsp://camcantho123.vinaddns.com:554/rtsp/streaming?channel=01&subtype=0" -c:v copy -an -f segment -segment_time 600 -reset_timestamps 1 -strftime 1 "C:\Users\QuangCler\Desktop\datacrawl\cam_%Y%m%d_%H%M%S.mp4"
```

▶ Processing: cam_20250922_181102.mp4

📁 Saved: 65A32807.jpg

📁 Saved: 65A11824.jpg

📁 Saved: 71A06206.jpg

📁 Saved: 65A48979.jpg

📁 Saved: 65A37428.jpg

▶ Processing: cam_20250922_181122.mp4

📁 Saved: 64A06833.jpg

▶ Processing: cam_20250922_181140.mp4

▶ Processing: cam_20250922_181200.mp4

📁 Saved: 65A42177.jpg

📁 Saved: 72G193457.jpg

📁 Saved: 66A25265.jpg

📁 Saved: 50F04940.jpg

📁 Saved: 65A25293.jpg

Data Preprocessing

2 datasets are used:

- Detection Dataset
- OCR Dataset

```
Data/
├── images/
│   ├── train/
│   │   ├── img_001.jpg
│   │   ├── img_002.jpg
│   │   └── ...
│   ├── val/
│   │   ├── img_101.jpg
│   │   └── ...
│   └── test/
│       ├── img_201.jpg
│       └── ...
├── labels/
│   ├── train/
│   │   ├── img_001.txt
│   │   ├── img_002.txt
│   │   └── ...
│   ├── val/
│   │   ├── img_101.txt
│   │   └── ...
│   └── test/
│       ├── img_201.txt
│       └── ...
└── dataset.yaml
```



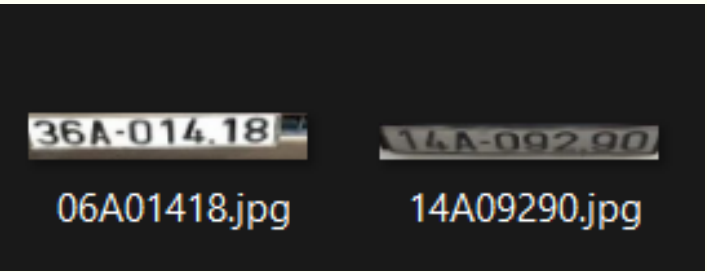
labels data

```
CarLongPlate936.txt
File Edit View
0 0.352754 0.712871 0.205508 0.092409
```

dataset.yaml

```
# /content/drive/MyDrive/Biển số xe/dataset.yaml
train: /content/drive/MyDrive/Biển số xe/images/train
val: /content/drive/MyDrive/Biển số xe/images/val
names: ["license_plate"]
```

```
OCR_LicensePlate_Dataset/
├── train/
│   ├── images/
│   │   ├── 0001.jpg
│   │   ├── 0002.jpg
│   │   └── ...
│   └── train_annotations.csv
│       └── (image_path, plate_text)
├── valid/
│   ├── images/
│   │   ├── 1001.jpg
│   │   ├── 1002.jpg
│   │   └── ...
│   └── valid_annotations.csv
│       └── (image_path, plate_text)
└── test/
    ├── images/
    │   ├── 2001.jpg
    │   ├── 2002.jpg
    │   └── ...
    └── test_annotations.csv
        └── (image_path, plate_text)
```



image_path	plate_text
images/05A25121.jpg	65A25121
images/06A01418.jpg	36A01418
images/14A09290.jpg	14A09290

Mô hình FastPlate sẽ đọc cặp ảnh + text để học cách chuyển ảnh → chuỗi ký tự.

Preprocessing Steps



Resize all images to 640×640. (For Yolo Detect)

For Finetuned-OCR:

Clean data: remove blurred or occluded plates.

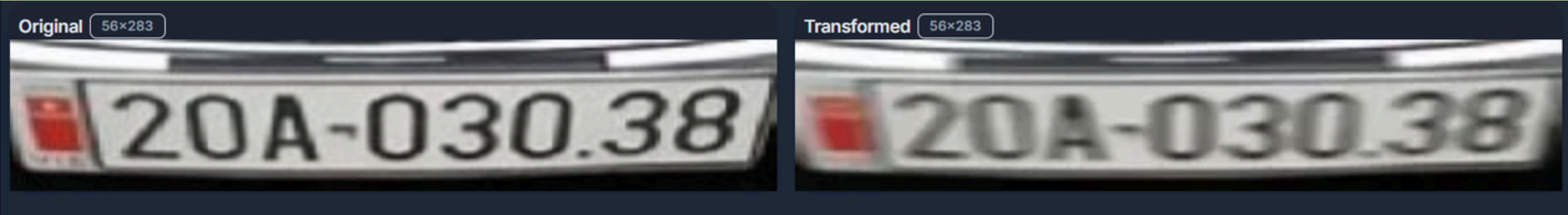
Split sets: Train 80%, Val 10%, Test 10%

Augmentation:

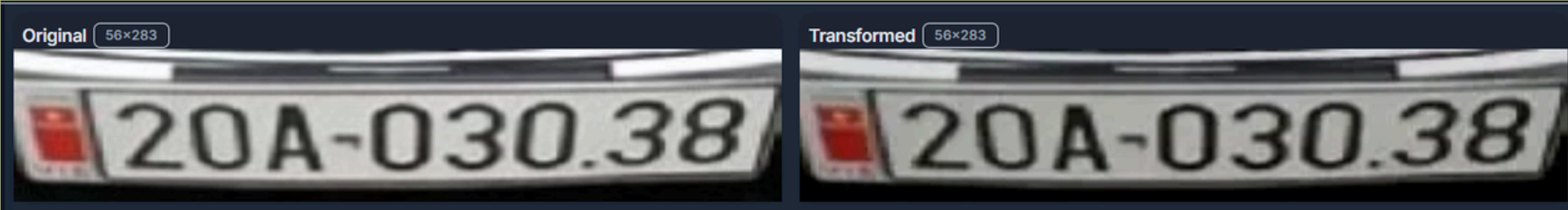
- Brightness & Contrast Adjustment
- Motion Blur
- Coarse Dropout
- HorizontalFlip
- ShiftScleRotate
- ISONoise
- ColorJitter
- ShiftScaleRotate
- ToGray

Preprocessing Steps

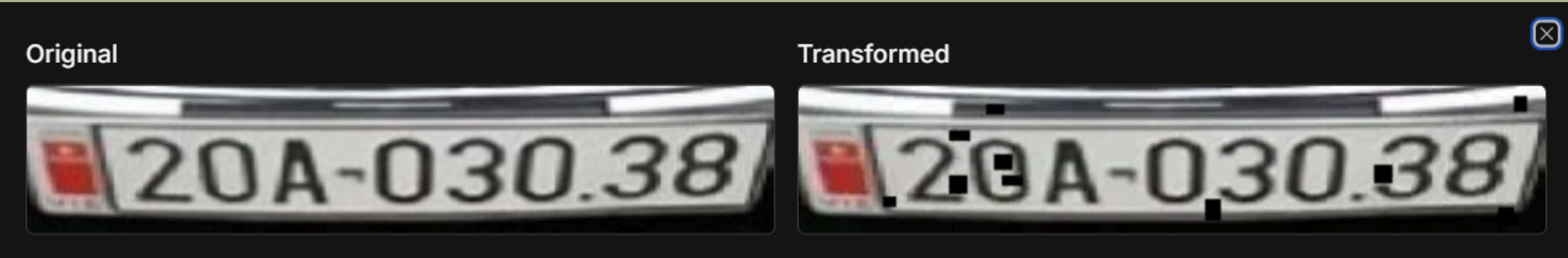
Augmentation Example:



Motion Blur: Blur limit (5,9)



Random Brightness Contrast



CoarseDropout



ColorJitter

Data Visualization

Data Distribution

Detection dataset:

- Most images: cars and motorbikes in daylight.
- Aspect ratio of plates: mainly between 3:1–4:1.

OCR dataset:

- Plate text length: 6–8 characters.
- Common confusion pairs: “B” ↔ “8”, “O” ↔ “0”, “0” ↔ “6”, “1” ↔ “7”

Finetune OCR dataset

Validation Summary

Category	Count
Errors	0
Warnings	1

Warnings

Line	Message
-	Character(s) not found in any plate: I, J, O, Q, W

Validation Summary

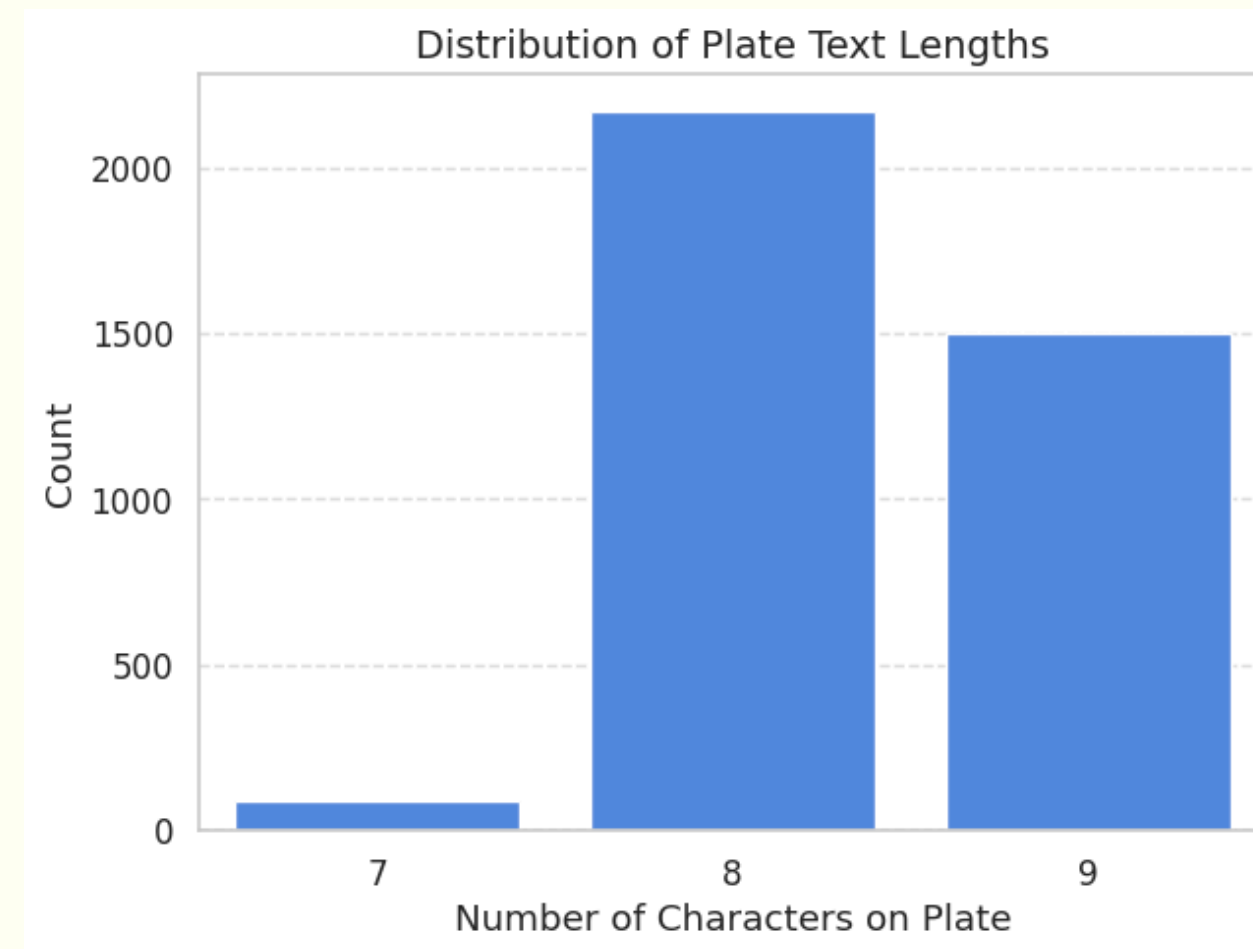
Category	Count
Errors	0
Warnings	1

Warnings

Line	Message
-	Character(s) not found in any plate: I, J, O, Q, S, W

Data Visualization

	plate_text	plate_length
0	05A25121	8
1	06A01418	8
2	14A09290	8
3	14A19068	8
4	14A43441	8



Most license plates contain 8 characters, dominating over 7- or 9-character plates.

This matches the standard Vietnamese plate format, confirming that the dataset is consistent and realistic.

Data Visualization

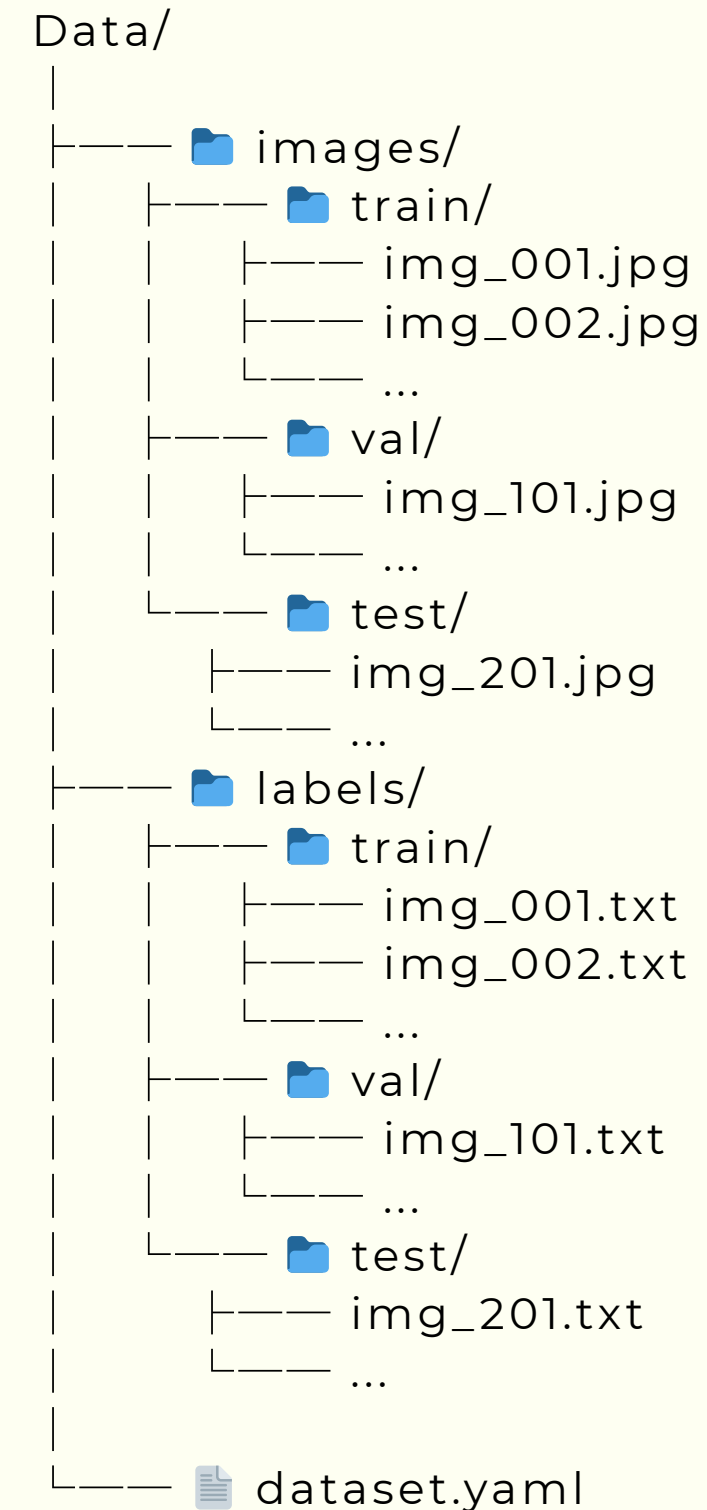
Dataset Statistics							
Plate Lengths							
count	mean	std	min	max	5%	50%	95%
3763.00	8.38	0.53	7.00	9.00	8.00	8.00	9.00
Image Height							
count	mean	std	min	max	5%	50%	95%
3763.00	114.08	49.18	21.00	1040.00	40.00	108.00	183.00
Image Width							
count	mean	std	min	max	5%	50%	95%
3763.00	157.49	72.06	30.00	841.00	67.00	133.00	299.00
Aspect Ratio							
count	mean	std	min	max	5%	50%	95%
3763.00	1.67	1.35	0.51	10.73	0.72	1.38	4.93

	Character	Count
7	5	4056
3	1	3691
6	6	3223
9	9	2741
1	0	2690
0	3	2459
10	2	2407
4	4	2307
11	8	2004
5	7	1979
12	A	750
2	F	429
21	B	324
14	H	260
13	C	253
26	P	226
29	G	197
15	K	182
24	L	179
30	S	146
18	D	139
8	E	136
20	T	120
22	M	105
25	N	104
28	X	102
16	U	91
17	V	82
19	Z	51
27	R	47
23	Y	39

License Plate Detection Model

Dataset structure:

- Train: Data/images/train
- Validation: Data/images/val
- Annotation format: YOLO .txt with bounding boxes.
- Total classes: 1 (license_plate).



License Plate Detection Model

	from	n	params	module	arguments
	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
	-1	1	6640	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
	-1	1	26080	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]
	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
	-1	1	87040	ultralytics.nn.modules.block.C3k2	[128, 128, 1, True]
	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
	-1	1	346112	ultralytics.nn.modules.block.C3k2	[256, 256, 1, True]
	-1	1	164608	ultralytics.nn.modules.block.SPPF	[256, 256, 5]
	-1	1	249728	ultralytics.nn.modules.block.C2PSA	[256, 256, 1]
	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
	-1	1	111296	ultralytics.nn.modules.block.C3k2	[384, 128, 1, False]
	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
	-1	1	32096	ultralytics.nn.modules.block.C3k2	[256, 64, 1, False]
	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
	[-1, 13]	1	0	ultralytics.nn.modules.conv.Concat	[1]
	-1	1	86720	ultralytics.nn.modules.block.C3k2	[192, 128, 1, False]
	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
	[-1, 10]	1	0	ultralytics.nn.modules.conv.Concat	[1]
	-1	1	378880	ultralytics.nn.modules.block.C3k2	[384, 256, 1, True]
	[16, 19, 22]	1	430867	ultralytics.nn.modules.head.Detect	[1, [64, 128, 256]]

O11n summary: 181 layers, 2,590,035 parameters, 2,590,019 gradients, 6.4 GFLOPs

The YOLO11n model consists of three main components:

- **Backbone:** Extracts visual features using Conv, C3, and SPPF blocks.
- **Neck (Feature Pyramid Network):** Combines multi-scale features through Upsample and Concat layers to improve detection of objects of different sizes.
- **Head (Detect):** Generates final predictions including bounding boxes, confidence scores, and class labels.

Lightweight design with 2.59M parameters, optimized for high speed and real-time performance.

License Plate Detection Model

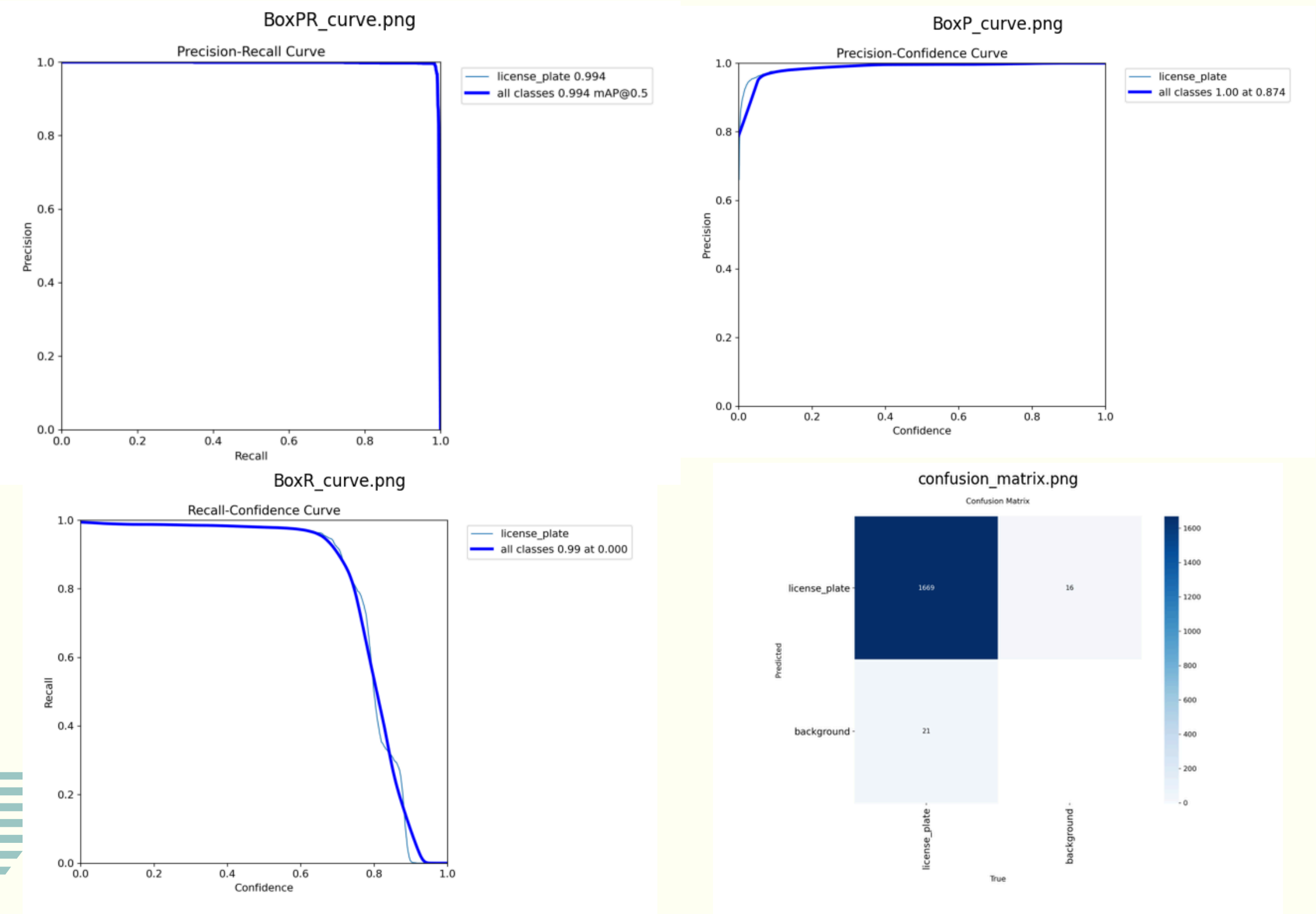
metrics/precision(B): 0.9956214424037426
metrics/recall(B): 0.9846153846153847
metrics/mAP50(B): 0.9937778779086472
metrics/mAP50-95(B): 0.7519962598922725
fitness: 0.7519962598922725

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
1/20	2.43G	1.159	1.055	1.065	25	640: 100%	413/413 5.71t/s 1:12<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 6.41t/s 0.15s0.1s
	all	1651	1690	0.963	0.949	0.969	0.625
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
2/20	2.68G	1.201	0.7418	1.094	31	640: 100%	413/413 5.91t/s 1:10<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.41t/s 7.0s0.1s
	all	1651	1690	0.972	0.941	0.972	0.645
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
3/20	2.69G	1.196	0.6653	1.092	18	640: 100%	413/413 6.11t/s 1:08<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.61t/s 6.9s0.1s
	all	1651	1690	0.98	0.966	0.983	0.664
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
4/20	2.71G	1.17	0.6209	1.077	29	640: 100%	413/413 6.11t/s 1:08<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.51t/s 7.0s0.1s
	all	1651	1690	0.975	0.962	0.983	0.681
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
5/20	2.72G	1.148	0.59	1.069	27	640: 100%	413/413 6.11t/s 1:08<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.51t/s 7.0s0.1s
	all	1651	1690	0.982	0.98	0.989	0.692
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
6/20	2.73G	1.135	0.5683	1.06	32	640: 100%	413/413 6.11t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.61t/s 6.8s0.1s
	all	1651	1690	0.98	0.978	0.988	0.702
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
7/20	2.75G	1.124	0.5447	1.056	32	640: 100%	413/413 6.11t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.61t/s 6.8s0.1s
	all	1651	1690	0.992	0.972	0.99	0.699
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
8/20	2.77G	1.104	0.5293	1.046	23	640: 100%	413/413 6.11t/s 1:08<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.51t/s 6.9s0.1s
	all	1651	1690	0.991	0.978	0.991	0.705
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
9/20	2.78G	1.096	0.5131	1.041	29	640: 100%	413/413 6.11t/s 1:08<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.51t/s 7.0s0.1s
	all	1651	1690	0.987	0.98	0.992	0.706
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
10/20	2.79G	1.107	0.512	1.047	29	640: 100%	413/413 6.11t/s 1:08<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.61t/s 6.8s0.1s
	all	1651	1690	0.994	0.979	0.993	0.715
Closing dataloader mosaic							
augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))							

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
11/20	2.81G	1.076	0.4733	1.057	15	640: 100%	413/413 6.11t/s 1:08<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.61t/s 6.8s0.1s
	all	1651	1690	0.992	0.977	0.993	0.711
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
12/20	2.82G	1.07	0.4621	1.051	15	640: 100%	413/413 6.21t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.61t/s 6.8s0.1s
	all	1651	1690	0.99	0.983	0.993	0.717
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
13/20	2.84G	1.06	0.449	1.054	15	640: 100%	413/413 6.21t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.61t/s 6.9s0.1s
	all	1651	1690	0.994	0.979	0.994	0.721
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
14/20	2.85G	1.051	0.4421	1.044	15	640: 100%	413/413 6.21t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.71t/s 6.7s0.1s
	all	1651	1690	0.995	0.977	0.992	0.736
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
15/20	2.87G	1.04	0.4239	1.036	15	640: 100%	413/413 6.21t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.61t/s 6.8s0.1s
	all	1651	1690	0.992	0.982	0.994	0.732
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
16/20	2.88G	1.033	0.4162	1.041	15	640: 100%	413/413 6.21t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.61t/s 6.8s0.1s
	all	1651	1690	0.992	0.982	0.993	0.733
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
17/20	2.9G	1.023	0.4065	1.032	15	640: 100%	413/413 6.21t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.71t/s 6.8s0.1s
	all	1651	1690	0.992	0.982	0.994	0.746
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
18/20	2.91G	1.016	0.3932	1.025	15	640: 100%	413/413 6.21t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.81t/s 6.7s0.1s
	all	1651	1690	0.994	0.983	0.994	0.74
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
19/20	2.93G	1.006	0.3855	1.018	15	640: 100%	413/413 6.21t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.71t/s 6.8s0.1s
	all	1651	1690	0.995	0.984	0.994	0.745
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
20/20	2.94G	0.9981	0.374	1.019	15	640: 100%	413/413 6.21t/s 1:07<0.2ss
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 52/52 7.61t/s 6.8s0.1s
	all	1651	1690	0.995	0.985	0.994	0.752
20 epochs completed in 0.416 hours.							
Optimizer stripped from /kaggle/working/runs/detect/train2/weights/last.pt, 5.5MB							
Optimizer stripped from /kaggle/working/runs/detect/train2/weights/best.pt, 5.5MB							

- Accurate and stable detection, suitable for real-world deployment.
- Trained for 20 epochs, well-converged with no overfitting.

License Plate Detection Model



License Plate Detection Model

val_batch0_pred.jpg



License Plate Detection Model



License Plate Detection Model

val_batch2_pred.jpg



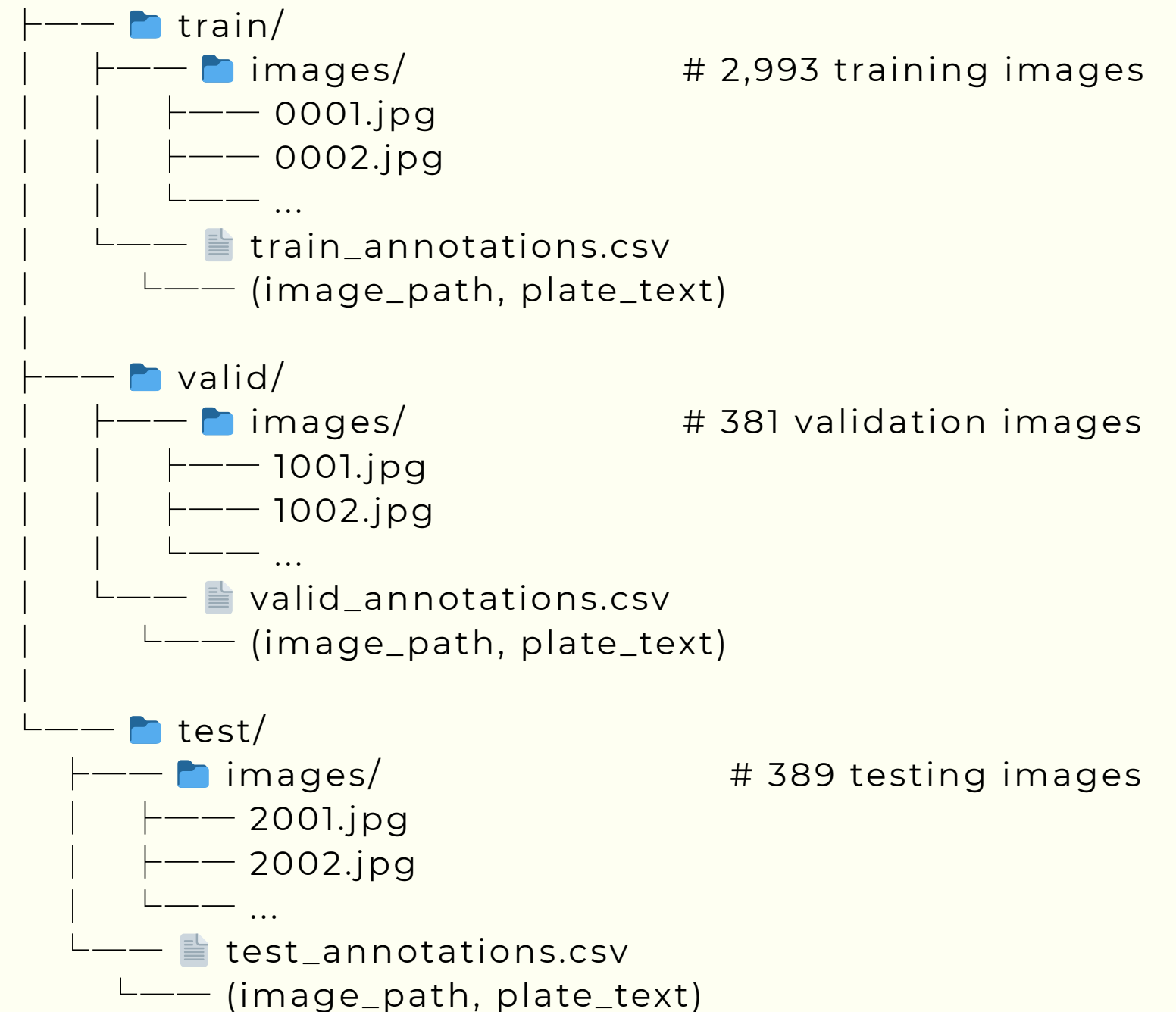
License Plate Recognition Model

Dataset structure:

Total samples: 3,763 images

- Train: 2,993 ($\approx 79.54\%$)
- Validation: 381 ($\approx 10.12\%$)
- Test: 389 ($\approx 10.34\%$)

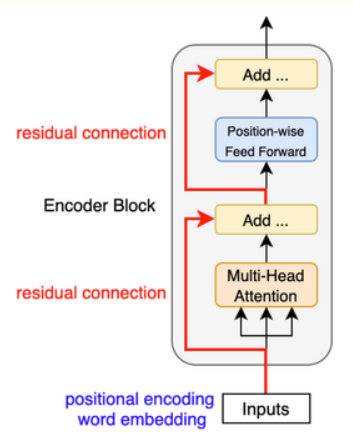
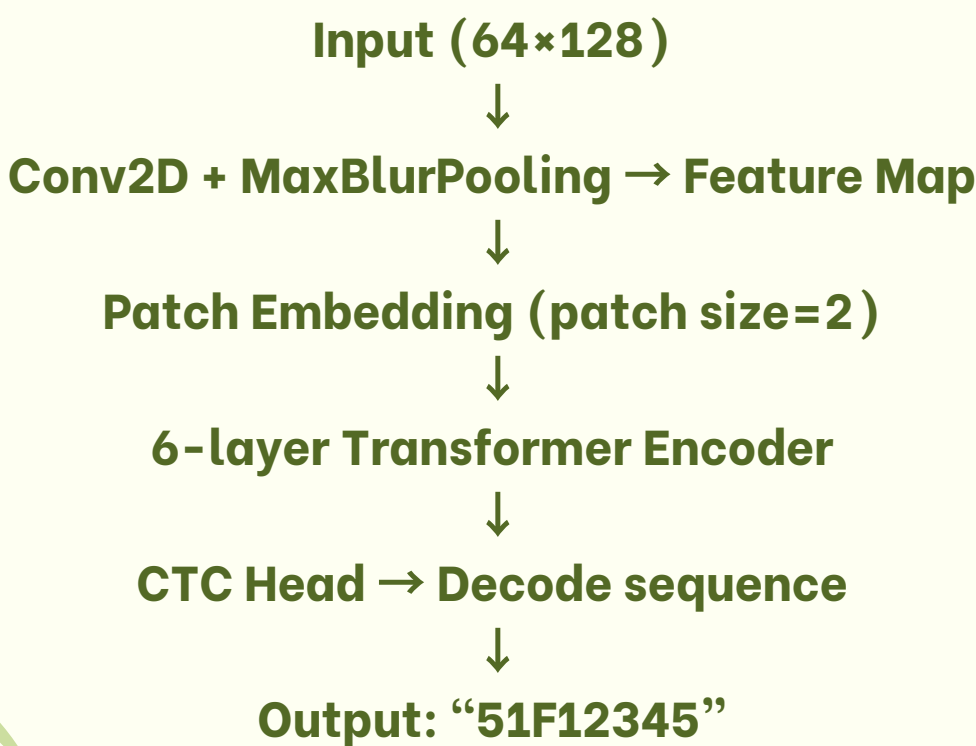
OCR_LicensePlate_Dataset/



License Plate Recognition Model

Main Components:

- **Backbone (Encoder):** Extracts visual features from license plate images (e.g., 64×128 RGB) using lightweight CNN or CCT blocks optimized for horizontal text.
- **Head (CTC Classifier):** Predicts character sequences with CTC loss, removing the need for explicit segmentation.
- **Uses CTC decoding** (greedy or beam search) with post-processing to remove duplicates and blanks (_).



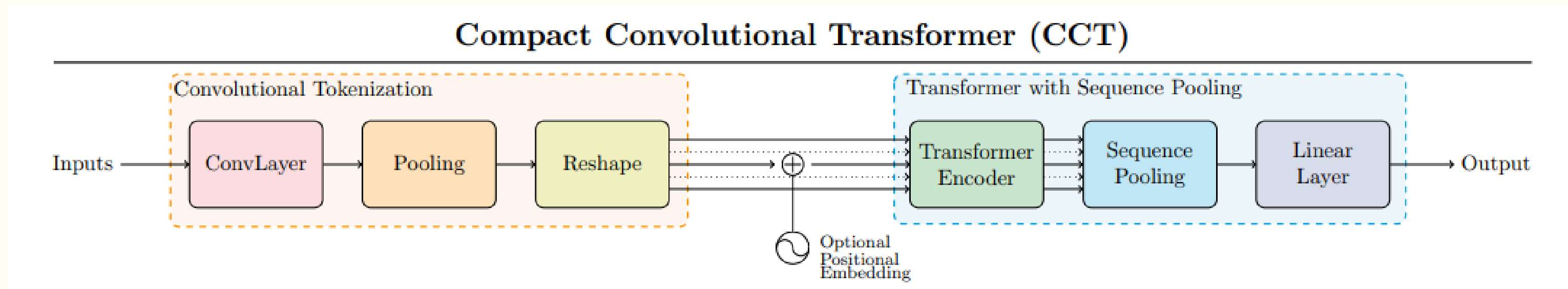
Main Component	Purpose / Role	Key Configuration
Input & Preprocessing	Normalize input images to ensure consistent size and ratio	- Input: RGB, 64×128 Rescale = 1/255 - Resize: linear, no aspect ratio kept
Backbone – CCT Encoder	Extract horizontal visual features from license plate images	- 5× Conv2D (48→80→96→128, GELU) + MaxBlurPooling2D - Patch size = 2 - Positional embedding = True - Transformer: 6 layers / 2 heads - Projection_dim = 128 - Dropout = 0.1–0.15
Head – CTC Classifier & Decoder	Predict and align character sequences automatically	- Alphabet: 0–9, A–Z, _ (32 classes) except (I,O,J,Q,W) - Max plate slots = 9 - Pad char = _ - Loss: CTC - Decoding: greedy / beam search
Output	Return final license plate string	- Example: "51F12345" - Post-processing: remove blanks, collapse repeats

Compact Convolutional Transformers (CCT)

CCT combine CNNs and Transformers to get the best of both worlds:

- Conv layers extract local features (edges, strokes).
- Transformer layers capture global context (relationships across the image).
- Positional embedding keeps spatial order.
- Compact design → fewer parameters, faster inference.

=> CCT = CNN for details + Transformer for context – fast, accurate, and lightweight.



Connectionist Temporal Classification(CTC)

- CTC lets the model read full license plate text without manual character segmentation.
- The CCT Encoder converts the cropped plate into a sequence of feature vectors along the horizontal axis.
- The CTC Classifier predicts one character (or blank _) per timestep.
- CTC Loss automatically aligns predictions with the correct text during training.
- At inference, the CTC Decoder merges duplicates and removes blanks → final output text.

Predictions: 5 5 _ 1 _ 1 F _ 1 2 3 3 4 5 _

→ Decode →  "51F12345"

License Plate Recognition Model

CLI Training Parameters	
Parameter	Details
model_config_file	'/content/drive/MyDrive/finefune OCR/cct_s_v1_global_model_config.yaml'
plate_config_file	'/content/drive/MyDrive/finefune OCR/cct_s_v1_global_plate_config.yaml'
annotations	'/content/drive/MyDrive/finefune OCR/test2/train_annotationins.csv'
val_annotations	'/content/drive/MyDrive/finefune OCR/test/valid.csv'
validation_freq	1
augmentation_path	'/content/custom_day_night_augmentation.yaml'
lr	0.0005
final_lr_factor	0.01
warmup_fraction	0.05
weight_decay	0.001
clipnorm	1.0
loss	'cce'
focal_alpha	0.25
focal_gamma	2.0
label_smoothing	0.05
mixed_precision_policy	None
batch_size	64
workers	1
use_multiprocessing	False
max_queue_size	10
output_dir	'trained_models'
epochs	50
tensorboard	True
tensorboard_dir	'tensorboard_logs'
early_stopping_patience	100
early_stopping_metric	'val_plate_acc'
weights_path	'/content/drive/MyDrive/finefune OCR/cct_s_v1_global'
use_ema	True
wd_ignore	'bias,layer_norm'
seed	None

Configuration:

```
{
    'max_plate_slots': 9,
    'alphabet': '0123456789ABCDEFGHIJKLMNPRSTUVXYZ_',
    'pad_char': '_',
    'img_height': 64,
    'img_width': 128,
    'keep_aspect_ratio': False,
    'interpolation': 'linear',
    'image_color_mode': 'rgb',
    'padding_color': (114, 114, 114),
    'vocabulary_size': 32,
    'pad_idx': 31,
    'num_channels': 3
}
```

Metrics for evaluation - Base model

GT: 59586X15 | File: 86X15955



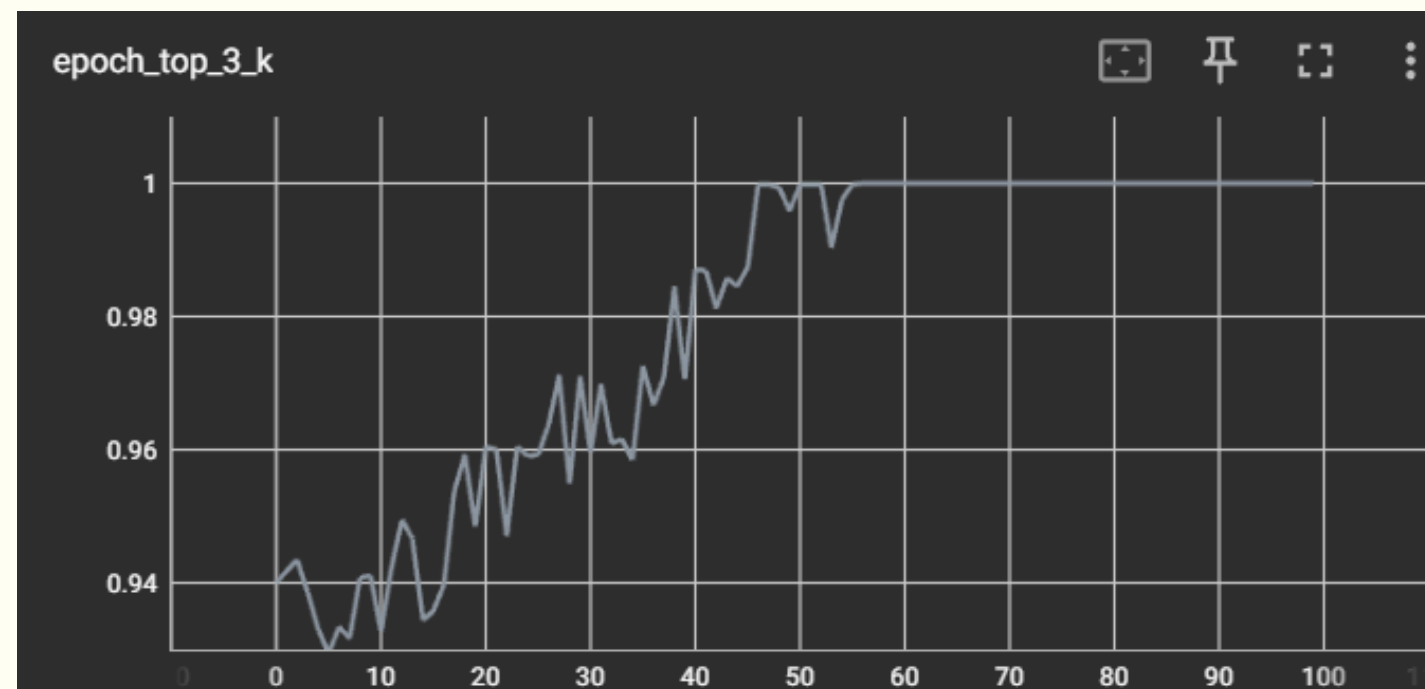
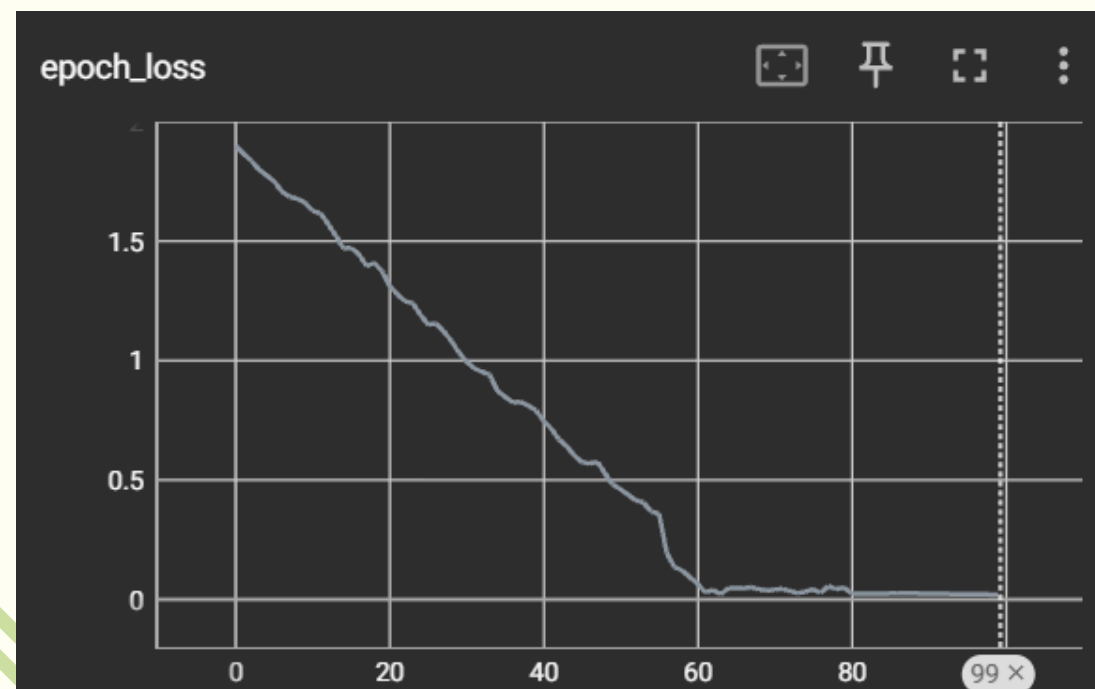
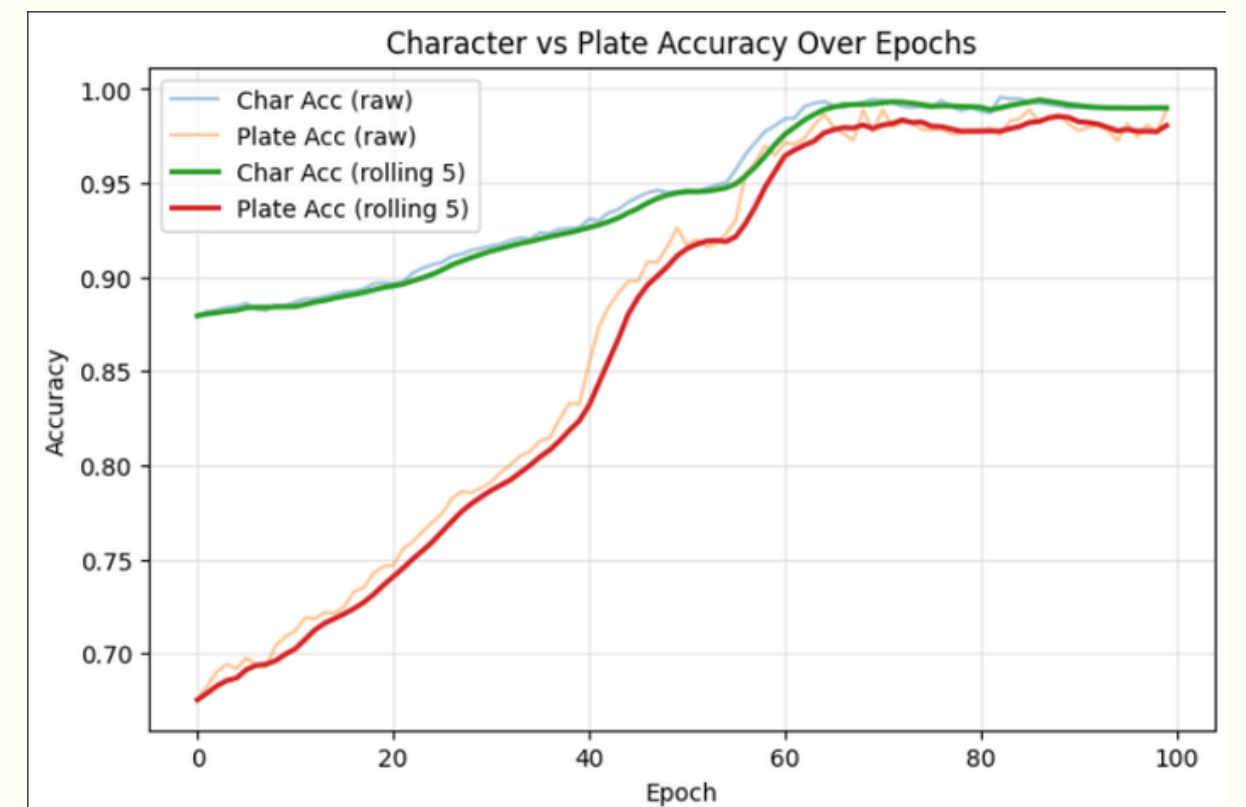
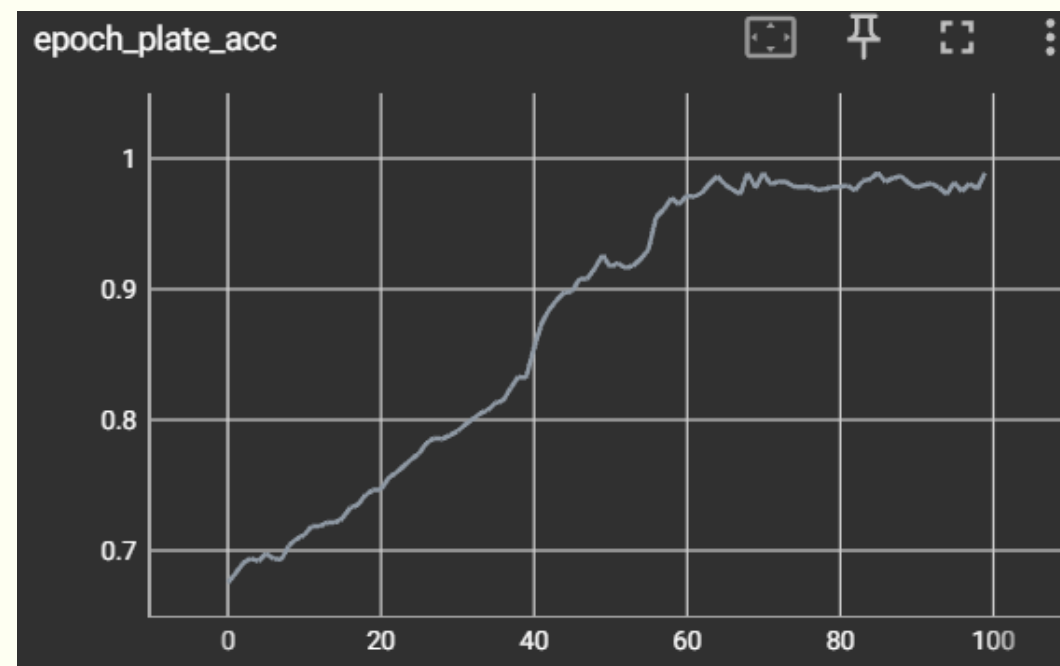
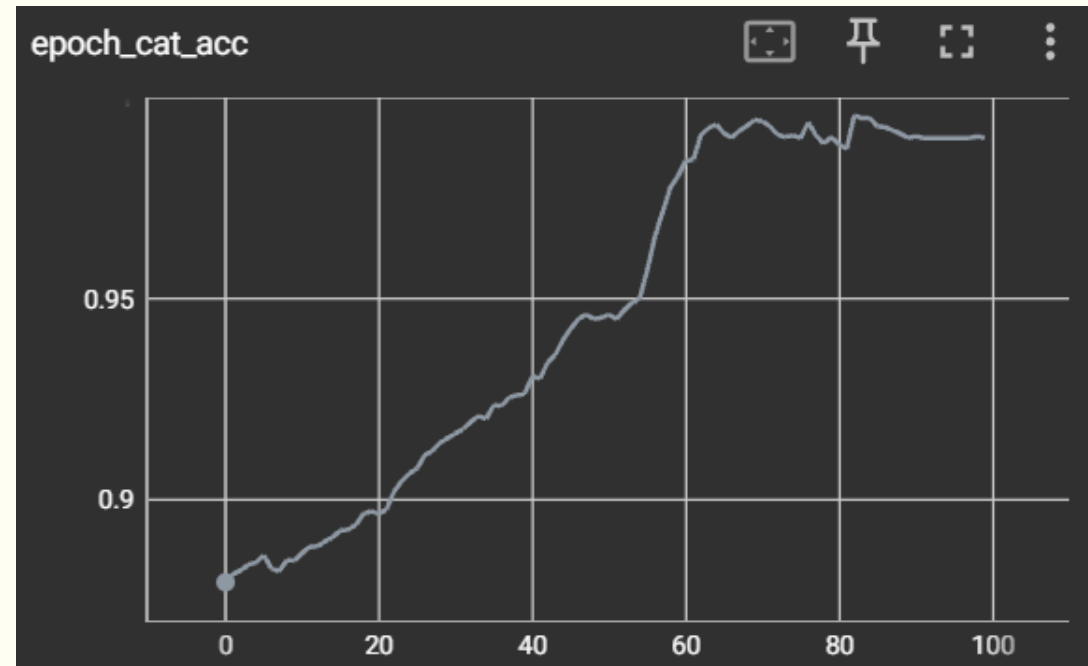
🔍 Most frequent character mismatches:

'7'	→	'1'	:	44 times
'1'	→	'7'	:	41 times
'I'	→	'1'	:	8 times
'J'	→	'1'	:	7 times
'Q'	→	'0'	:	7 times
'5'	→	'1'	:	6 times
'5'	→	'6'	:	6 times
'4'	→	'5'	:	5 times
'5'	→	'7'	:	5 times
'5'	→	'0'	:	5 times
'1'	→	'8'	:	5 times
'1'	→	'5'	:	5 times
'2'	→	'5'	:	5 times
'6'	→	'1'	:	5 times
'0'	→	'1'	:	4 times

Base model

- ***cat_acc: 0.8794***
- ***loss: 1.8977***
- ***plate_acc: = 0.6752***
- ***plate_len_acc: 0.9778***
- ***top_3_k: 0.9118***

Metrics for evaluation - Base



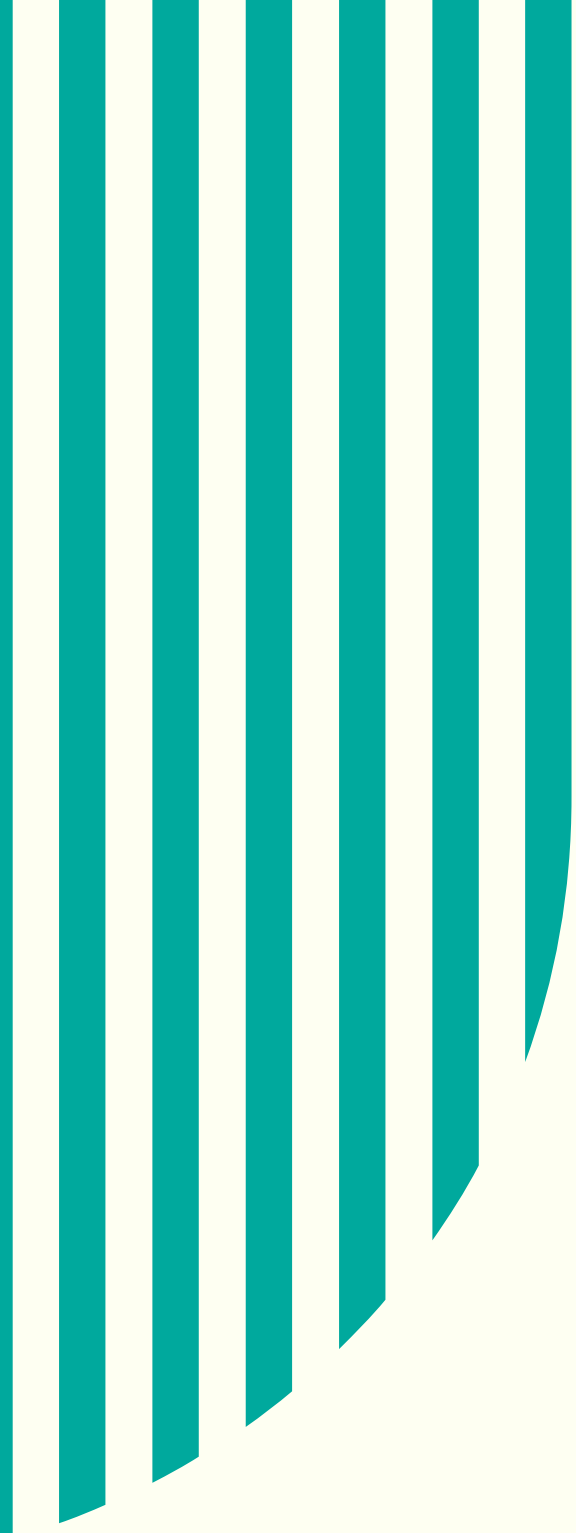
Metrics for evaluation Results

Metric	Base	Fine-tuned	Improvement	Relative Gain
Character Accuracy	0.8794	0.9899	0.1105	×1.13
Plate Accuracy	0.6752	0.989	0.3138	×1.46
Loss	1.8977	0.0205	-1.8772	↓98.9 %
Plate Length Accuracy	0.9778	1	0.0222	×1.02
Top-3 @ K Accuracy	0.9118	1	0.0882	×1.10

- **Character Accuracy increased significantly, reaching nearly 99%, indicating that the OCR character recognition capability is now extremely precise.**
- **Plate Accuracy improved by more than 1.46×, demonstrating that the model has effectively learned the overall structure of license plates.**
- **Loss decreased by approximately 99%, reflecting strong convergence and reduced gradient noise.**
- **Both Top-3 @ K and Plate Length Accuracy achieved 100%, confirming the fine-tuned model's reliability and stability.**

Conclusion:

- *Dataset refined by removing unused characters (I, J, O, Q, W) → higher data consistency.*
- *Base model struggled under varied image conditions → Albumentations improved robustness.*
- *Model learns horizontal plate layouts effectively (no row separation needed).*
- *Still performs well on two-line plates thanks to CTC-based sequence learning.*
- *Fine-tuning delivers strong accuracy and real-world adaptability.*



Group 2

THANK YOU

